Architecture = Abstractions over Software*

Eyðun Eli Jacobsen Bent Bruun Kristensen Palle Nowack
The Maersk Mc-Kinney Moller Institute for Production Technology
University of Southern Denmark/Odense University, DK-5230 Odense M, Denmark
e-mail: {jacobsen, bbkristensen, nowack}@mip.ou.dk

Abstract

Design of software architecture is seen as abstraction over the software domain, and describing architecture is considered to be a modeling process. A general view of a modeling process is presented and illustrated in the context of application domain modeling and of software domain modeling. The implications of this perspective are investigated in order to capture objectives and concrete forms of architectural descriptions. The consequences of this perspective on architecture are characterized.

1: Introduction

In every software system, some architecture is present. The problem is that architecture is only implicitly available. The architecture did exist for developers during the design phase, but because it was not expressed explicitly, the architecture either has been lost, or has only been reproduced to some extent in software documentation. The consequences of the architecture are present in terms of the properties and qualities of the software.

Software Architecture. Software architecture is defined in various different ways.
[9]: "Abstractly, software architecture involves the description of elements from which systems are built, interactions among those elements, patterns that guide their composition, and constraints on these patterns. In general, a particular system is defined in terms of a collection of components and interactions among those components. Such a system may in turn be used as a (composite) element in a larger system design."
[3]: "A software architecture is a description of the subsystems and components of a software system and the relationships between them. Subsystems and components are typically specified in different views to show the relevant functional and non-functional properties of a software system. The software architecture of a system is an artifact. It is the result of the software design activity."
[10]: "An architecture integrates separate but interfering issues of a system, such as provisions for independent evolution and openness combined with overall reliability and performance requirements. An architecture defines guidelines that together help to achieve the overall targets without having to invent ad hoc compromises during system composition."
[1]: "The software architecture of a program or computing system is the structure or structures, which comprise software components, the externally visible properties of those components, and the relationships among them."
[8]: "Architecture means style or manner of building". For a software system we interpret software architecture to mean the actual choice of architectural abstractions and language mechanisms, especially abstraction mechanisms, and the actual use of the elements chosen, i.e. how they are combined, their iterations, variations etc."

In a brief characterization of these definitions of software architecture we claim that these describe organization only, not abstraction. We distinguish between organizing and abstracting: If we organize a certain

*This research was supported in part by Danish National Centre for IT Research, Project No. 74.

domain, we only put some kind of order in the domain, among the elements in the given domain. The result is that the elements are grouped, sorted, or otherwise related to each other according to different possibly overlapping organization principles. If we abstract from a certain domain, (may be in general for abstraction and not only conceptual abstraction) we create new "concepts"/elements in another domain, an abstract domain. The elements in the new domain describe the elements in the former domain, and the relations among elements in the former domain. By moving to another domain we create knowledge about the former domain, and not just order. The concepts/elements of the new domain carry information about the former domain.

The architecture of some software is a model of the software. In this article we focus on conceptual models of software. A model is conceptual in the sense that it forms a conceptual understanding of something, and that it is based on concept formation in terms of classification, generalization and aggregation.

Consequences: Abstractions over Software. Our thesis is that software architecture is abstractions over the software domain. Some immediate consequences of this thesis include that
(1) The software must have been produced or at least be anticipated, i.e. the software domain must exist in some form. The software domain is given by descriptions — focus during architecture design is on these descriptions. The result of design are concrete/abstract *descriptions* (notation/diagrams) in addition to the software domain. But there is no given sequence — architecture may be created as a *prescription* of some anticipated software.
(2) Architecture descriptions appears to be *redundant* descriptions. Because the software domain is available architecture design seems to be an additional and even redundant description. The challenge is to clarify why it is necessary to elaborate on the software description, to make an understanding / a mental model explicit, and to create abstractions over the software domain.
(3) Architecture design requires abstraction. Abstraction implies, in addition to organization and delimitation, formation of concepts at another abstraction level together with use of the concepts at that level. We abstract from irrelevant details and concrete matters, given selected perspectives. But we have to find, introduce, and motivate the abstractions (and the perspectives). Enough experience in architecture design and insight in the given application is necessary to be able to come up with suggestions for *the right abstractions*, and to decide which are the correct and promising ones.
(4) The nature of abstractions over the software domain is still partially unexplored and unfamiliar. In our abstractions in object-oriented modeling the descriptions are concrete — abstract matters are related to problem and the usage domains. Abstractions over the software domain are different — probably more abstract and unreal. The challenge is to clarify the *essence of software abstractions*, and specify the quality measure for abstractions.
(5) The architecture design process must be covered by *software development methodologies*. Architecture is only superficially introduced, usually only mentally, because of lack of support of this in methodologies. Architecture design *is* difficult, and because architecture design is not covered by methodologies the general situation is that we have little experience, no very good examples or prototypical cases. Methodologies do not promote (acceptance of) reuse of design — reuse of code is still the rule.

Article Organization. In section 2 we establish the necessary background for the perspective on software architecture to be discussed in section 3. In subsection 2.1 we briefly discuss a model of the domains and views in the design phase in the software development process. In subsection 2.2 we elaborate on an underlying model of abstraction processes in relation to software development processes. In section 3 we discuss consequences of architecture seen as abstractions over the software domain. We characterize the process-related and the notation-related aspects, and we propose principles governing architectural design. In section 4 we summarize the contributions of this article.

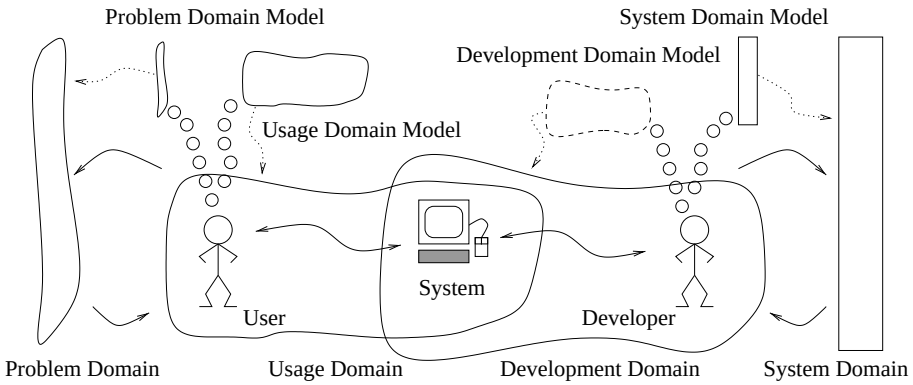

Figure 1. Domains and Models

2: Abstraction

2.1: Domains and Views

In Figure 1 we illustrate the domains and models from an abstract generalized model of the software development process (the domains and models discussed in the following are a revised and more complete model than the model from [5]). In the analysis phase the problem domain model and usage domain model are constructed from the problem domain and usage domain through a cooperation between user and developer. In the design phase the developer in the development domain uses the usage domain model to refine the problem domain model into a system domain model.

The domains and models illustrate the vision during the system development process — as such they reflect the understanding according to the current iteration. The focus point in the illustration is the system to be developed. The system illustrates the concrete system — the actual, real system.

The system is going to be used by some user. As such the system is part of the usage domain. The system takes care of some tasks in relation to the problem domain. The user has an understanding of the problem and usage domains — namely the models of these domains. When the user operates the system these models form the universe of the operations, their state and their results. The concrete form of the problem domain model and the usage domain model can be respectively [2]: object diagrams, class diagrams, use case diagrams, interaction (sequence and collaboration) diagrams, state chart diagrams, and activity diagrams.

The system is going to be developed by some developer. As such the system is part of the development domain. The developer is part of the development domain — the model of this domain is implicitly given only (that is why the model has dotted lines). The system domain is the actual resulting software that comprises the system at the very concrete level. The system domain is the actual software. The system illustrates the software *and* everything else that is needed to make it a running system. The system domain model forms the developer's conception of the integration of the problem and usage domain models at an abstract level. It is a refined, transformed and enriched problem domain model. The concrete form of the system domain model can be [2]: object diagrams, class diagrams, interaction (sequence and collaboration) diagrams, state chart diagrams, activity diagrams, component diagrams, deployment diagrams. Most of the diagramming techniques are the ones known from object-oriented analysis. When applying them to describe the system domain model we apply them differently: the objects and classes are of a more technical nature (not necessarily representing problem or usage domain phenomena and concepts); they belong to the software domain; they have more rigorously defined properties. As a result of the changed perspective on the objects and classes, the applications of diagrams in the description of a system domain model also has a more technical nature. In addition to these diagrams we can also specify or sketch designs in an object-oriented programming language. Depending on the goal of the description, the described code can act as a vehicle for communicating ideas or as a basis for a prototype implementation. The system domain model also supports the usage of the system, and not only the conception of the problem domain. The usage of the system is seen from two different points of view — the user's and the developer's.

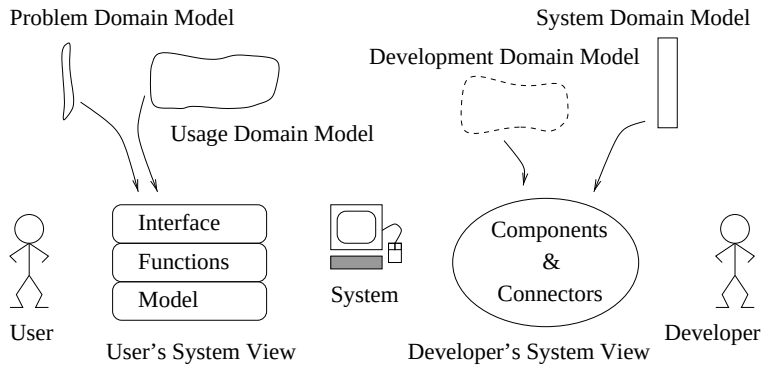

Figure 2. System Views

Figure 2 elaborates further on the distinct views on the system from the user and developer. The user view supports the understanding in terms of the problem and usage domains. The system operates on a model of the problem domain, based on some functions that support the identified usage patterns captured by the usage model, through an interface that exposes the functions according to the user's capabilities etc. The developers view supports the understanding in terms of the system domain model. This understanding is controlled and supported by the development domain (model). The system is organized as a collection of components and connectors of very general nature. The actual organization of these reflects the functional and non-functional requirements to the system[1].

The architecture model (Figure 3) is an abstraction over the system domain model. The architecture model forms the developer's conception of the architecture of the system, i.e. the overall structures and their relations and interactions. The model focuses for example on structure and interaction embedded in the system domain model. The purpose of the architecture model is to understand the system domain model given a various selected perspectives on that model, to allow us to reason about and expose the support of non-functional requirements in the software, and to map the system domain model onto the logical platform. The system domain model includes descriptions, partial or complete, of software. Various kinds of design notations are used.

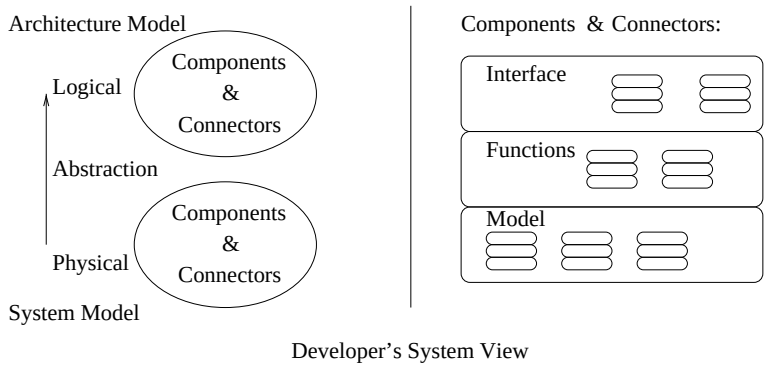

Figure 3. Abstraction and Components/Connectors

In Figure 3 the architecture model is shown as a logical model — an abstraction over the system domain model. We call the system domain model a physical model because this is how the model of the system

[1]The functional requirements is a set of requirements concerning the capabilities of a system — what the system should be able to support. The non-functional requirements is a set of requirements concerning qualities not related to the functionality of the system — for example usability, security, efficiency, correctness, reliability, maintainability, testability, flexibility, understandability, reusability, portability, adaptability. The logical platform is a description of the platform on which the system is going to be executed but at a logical level — i.e. requirements in terms of user interface, distribution, persistence and concurrency.

domain is represented. The physical model is the basis for various additional models — logical models, because they express various understandings of the system domain model. The logical models together form the architecture model. In Figure 3 the models at both levels are illustrated as collections of components and connectors. Components and connectors should not be seen literally. The idea is only that at a certain level any description of a model can be seen as some specialized kinds of components and connectors. Also we have indicated that both components and connectors can be seen as structures with three ingredients, namely interface, functions and model, where each of these are recursively collections of components and connectors.

The result of the design phase is not a model of some *existing* domain similar to the problem domain or the usage domain — the system domain is non- or only partial existing during the development process. Rather, the system domain model is a unique design, something constructed and created by the developer based on his skills and knowledge (the development domain), and from the problem and usage domain models. After its construction the system domain architecture models both works as models in addition to the problem and usage domain models, but from specific design perspectives.

The problem and system domain models are the foundation for creating the architecture. The usage model is used during the design for controlling the transformation of the problem domain model into the system domain model. The usage model, together with the functional requirements, supply the information for adding and distributing the functionality to the problem domain model to the extent that this information is not already there. The architecture model is a constructive solution for how the non-functional requirements and the organization of the logical platform can be combined.

2.2: Abstraction and Modeling.

The Nature of Abstraction. We discuss nature and purpose of abstraction — both in general, abstract terms and in terms specific to abstraction over the software domain.

We distinguish between referent system and model system. The referent system is the domain we observe — the "real" world (even though it may be imaginary). The model system is the product we construct in the form of some explicit representation of the abstract model. We observe phenomena in the referent system, and we classify these by concepts also in the referent system. Figure 4 illustrates the referent and model systems. It includes the modeler and the perspective applied on the real world. The connections between the elements in the domains in the referent system and the model system are indicated by means of dotted lines.

A number of more or less well defined notions within abstraction exist for concept formation in the referent system. These include among others classification, generalization and aggregation (and dual or inverse notions of these). When we observe phenomena in the referent system we always apply a perspective. The perspective determines which properties we choose to include in our abstract model — properties form concepts.

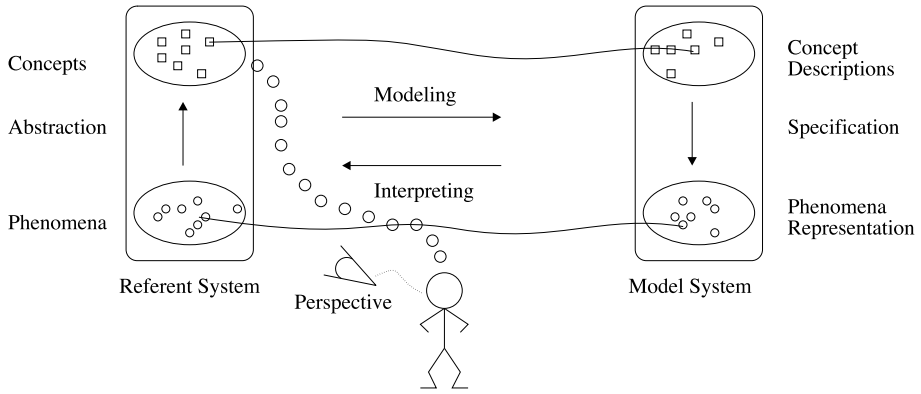

Figure 4. Worlds, Perspectives & Models

The purpose of abstraction is to understand. If we do not abstract, we only see a wilderness of different

phenomena in the referent system. Applying concepts, we put order in chaos, so that we can understand the referent system. The concepts expresses our understanding — but always relative to a chosen perspective. The model *describes* the referent system phenomena and concepts, but it is not the referent system. And it is not a mirror image of the referent system. The model is a simplification, but more important the model expresses only one out of many possible interpretations of the referent system. It is necessary to introduce simplifications, and to focus on certain aspects of the referent system, in order to expose the understanding given the perspective. Else understanding drowns in details irrelevant to a perspective.

Observation of phenomena is collective in the sense that several persons usually can agree upon that they observe some phenomena. However, conception of phenomena is subjective. Concepts formed mentally by observers are individual. By expressing the concepts explicitly in some notation they become collective in the sense that several people can discuss and understand the model — they share the understanding expressed through the model. The perceived concepts are subjective. The explicit model expressed in some notation may also be subjective, but it can be shared. Under given circumstances we can agree that the model is objective. That the model is objective, does not imply that it is fair or realistic. The perspective alone determines to which extent the model should reflect certain aspects of the referent system or not. It is not an overall goal to obtain any specific kind of correspondence between the model and the referent system. The success criterion for the model is to which extent it exposes the intention of the model as prescribed by the perspective.

Perspectives are not formal specifications. In practice perspectives are not even fixed during modeling. They are developed together with the model. There is always a more or less precise expectation to the intention of the model. It may be intuitively clear what is important and what is not. But as the modeling process evolves with various candidates for concepts and properties of these, the perspective becomes visible too. At the end of the process the best explanation of the perspective may be the model itself.

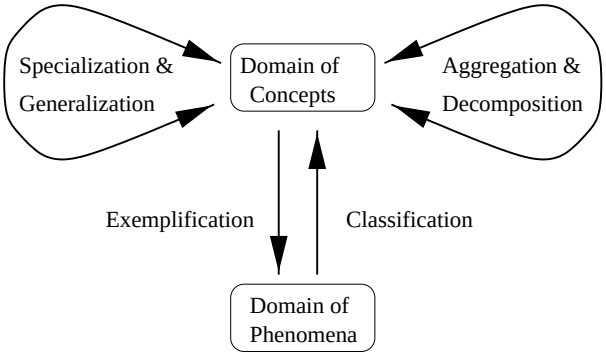

Figure 5. Abstraction Processes

We apply this general understanding of the nature and intention of abstraction when software architecture is seen as abstractions over software. The software is our referent system. We can all observe the software, and we can identify various kinds of phenomena in the software. We have to choose perspectives in order to guide the abstraction process.

We can observe software either as program description or as program execution. The latter is typically a sequence of snapshots — infinitely many in general. Executions are controlled by means of input to the execution in a broad sense. The program description captures (statically) all (dynamic) executions and thus all sequences of snapshots. Program description can be seen as an abstraction over sequences of snapshots — a classification of these. Sequences of snapshots can be seen as exemplification of the program description. This abstraction (program description) is different in nature from the architectural abstractions we discuss in this article. In short program description is obtained by classification of program executions (e.g. sequences of snapshots) — executions are obtained by exemplifying program description (by having a generator work on the description). Program description and execution are bound to the same domain — the same referent system.

Figure 5 illustrates the abstraction processes in conceptual modeling. Conceptual modeling can be seen as

the underlying theoretical understanding of object-oriented programming although existing object-oriented notation only in a limited way supports this theory. The processes are fundamental abstraction processes. In object-oriented programming an object/class can be seen as a model of a phenomenon/concept — to give the class of an object corresponds to classification — to give an object of a class to exemplification. To let a (specialized) class inherit from a (general) class corresponds to form a specialization of that (general) class. To describe a (whole) class by use of references to (part) objects corresponds to form an aggregation of (part) objects.

Software Development. Two significant and distinct application of abstraction can be identified in the software development process. In Figure 6 both these abstraction processes are shown and related.

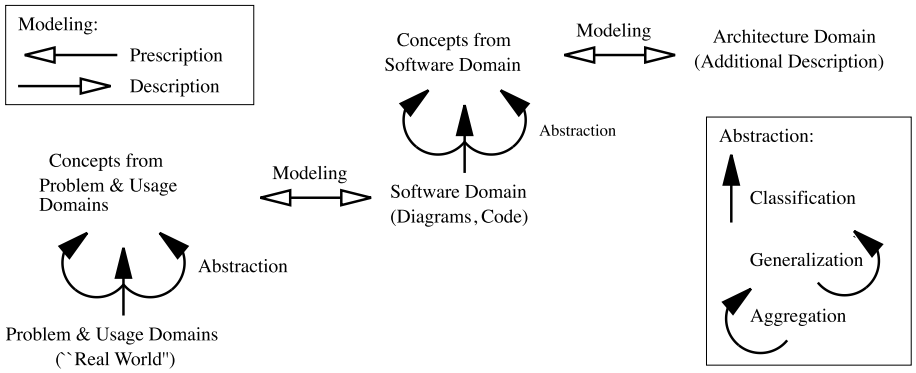

Figure 6. The Abstraction & Modeling Diagram

During analysis we build models in order to make explicit our understanding of the problem and usage domains. We describe phenomena and concepts from the problem and usage domains with diagrams and code from the software domain. The problem and usage domains form the reference system and the software domain form the model system.

During design we build models in order to make explicit our understanding of the envisioned software. We describe phenomena and concepts from the software domain with diagrams and additional notation from the architecture domain. The software domain forms the reference system and the architecture domain forms the model system.

The illustration should be considered static; we do not impose any ordering of the described processes. Any practical development effort alternates between analysis and design. Instead we focus on the involved domains and abstractions. Whether we perform analysis or design we experiment with different concepts (and thus phenomena) in order to come up with suitable models. Many issues constrain the validation of the suitability of the models. A constraint of vital importance in object-oriented development is that the models formed during analysis are reflected in the models formed during design.

To sum up Figure 6 contains two instances of the referent and model system — the model system of the left instance is also the referent system of the right instance. In the following we describe the two instances in more detail.

Figure 7 illustrates the analysis perspective on software development. During analysis we form concepts over the problem and usage domains. We model these with object-oriented entities. Classes model concepts and objects model phenomena. In our object-oriented model, classes can be considered to specify the objects. Once the model has been established we understand and interpret the referent system in terms of the identified objects.

Objects and classes are well-known mechanisms in notations for model systems. Other mechanisms such as references, subclassing, etc. are used to model notions from the referent system. In general a transformation is necessary from the referent system to the model system. In general notation lacks mechanisms that directly support a lot of natural conceptual notions from the mental model. Such notions must be simulated by the transformation. Efficient and understandable representations of such notions in the common object paradigm are important.

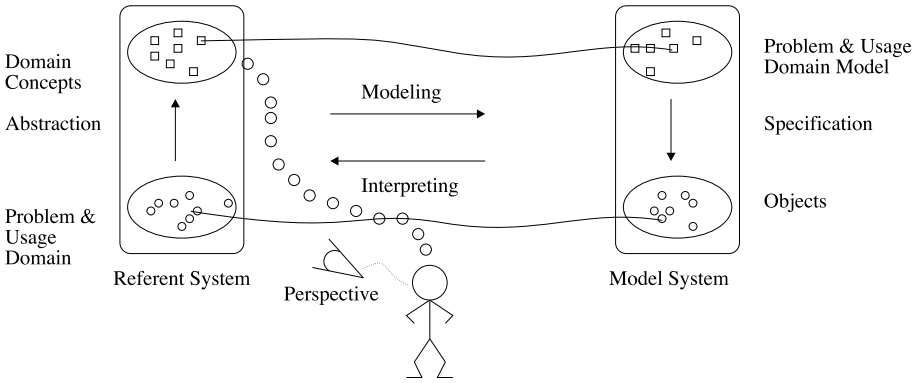

Figure 7. Analysis Phase: Functionality

Figure 8 illustrates the design perspective on software development. During design (and implementation) we form concepts over the software domain — such concepts do not categorize anything from the problem and usage domains (at least only indirectly). The software domain is the reference system. The model system explicitly describes the phenomena and concepts from the software domain. Many different perspectives can be applied to form a software model; we discuss only architectural models of the software domain.

Concepts formed over the software domain include design patterns [4], architectural patterns [3], components, connectors, and architectural styles [9]. Architectural descriptions seek to describe such concepts in a rigorous way. The descriptions specify representations of architectural phenomena, that facilitate the interpretation and understanding of the software domain. However, there is still no established conceptual framework for describing such concepts, and there is still a lack of support for describing architectural models.

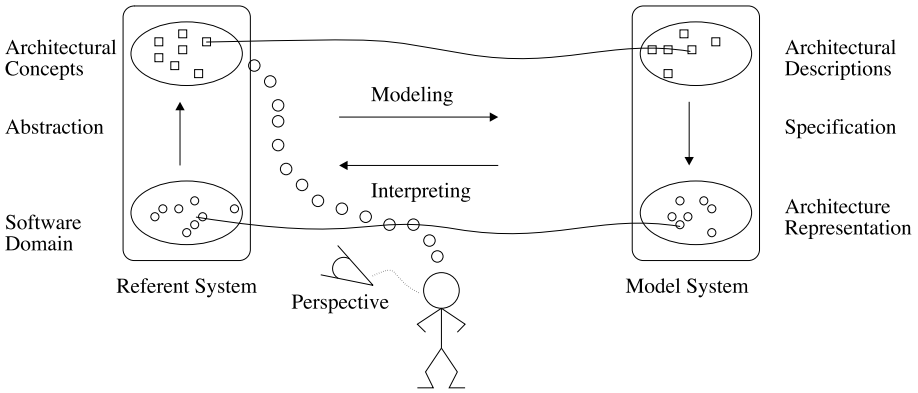

Figure 8. Design Phase: Architecture

In general the description of a model system is supported by our available language (or notation). The language reflects the conceptual framework through which we understand the model. The language can be formal or partially informal.

3: Architecture

Static and Dynamic Aspects. We discuss static and dynamic aspects of software architecture. One dimension of static and dynamic aspects is the distinction between (static) description of a system (the program) and (dynamic) execution of it (snapshots with state and state transitions).

Description and Execution. Architectural information is obtained by abstracting over either the description or the execution. In Figure 9 we illustrate the description to the left. The description is some representation of the program. It is static — at least we postpone the discussion of the case where the program changes during the execution of it. To the right we illustrate a sequence of snapshot of the execution of the program (description). Each snapshot has some state, and transitions (simple or compound) take the execution from one snapshot to the next. The transitions are not discussed further, the dynamics is represented only through the selected sequence of snapshots. The snapshots are seen as sequences (for simplicity reasons only) — tree structures reflect various alternatives during execution in a natural way.

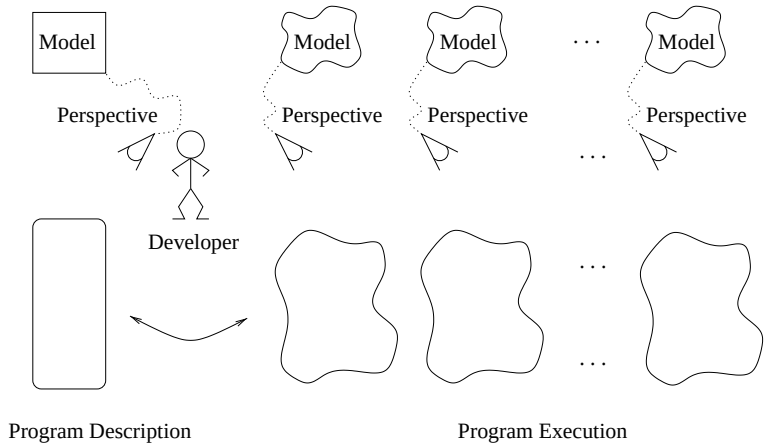

Figure 9. Description & Execution

For each snapshot the architecture can be described. By external stimuli, and as time goes by, the system makes transitions to a sequence of snapshots — each with an architecture. When we describe the architecture of a single snapshot we are describing the architecture of a static thing. But when we describe the architecture of a running program we are describing the relation between several snapshots.

Several types of information can be obtained by applying different perspectives for the abstraction. The same architectural abstraction can be observed from both the description and the execution, but it will be expressed differently. In the execution the interaction among objects and object structures are in focus, while in the description the interaction is embodied in methods and classes — furthermore, in the description, there are many language mechanisms which are use to express something about the execution of the program, but do not themselves exist at the time of execution, and these language mechanisms can be subjects for architectural descriptions.

3.1: Process.

The Description Process. First the interaction with the real world is defined. The system is treated as a black box. The execution is in focus. Next the overall system organization and interaction are defined. The system is treated as a white box. Still the focus is on the execution. From the understanding of the dynamic situations a description in the form of program/diagram is produced, together with abstract views of this.

Characterization: Process of Architecture Design.

- *Architecture design is innovative construction, not just modeling or transformation.* Design of architecture is not just modeling: There are no existing phenomena in the software domain to model (like in the analysis phase, where the developer can take the starting point in problem and usage domains). Design of architecture is not just transformation: There is no existing model available (like in implementation where the model resulting form the design has to be transformed into another model at programming language level). In practice innovative processes give problems both because they are difficult and expensive, and also because they are hard to control and administrate.

- *Architecture design implies abstraction, not just organization.* By organization we mean the process of arranging and relating subparts of wholes. This resembles aggregation and association to a large extent. Abstraction includes organization in the sense that chunks of substance have to be identified and delimited. But abstraction also involves classification and generalization — a given chunk is classified according to various existing or new concepts — the concept is related to other concepts. In addition the abstraction is thereafter treated at its abstract level. In practice it is straightforward to organize material. During that process we get some knowledge about the material. But by organization only we more or less just view information at the given level. To obtain important knowledge, we have to abstract. Abstraction is much more demanding than organization.

3.2: Notation.

Characterization of Styles & Patterns. It is possible to characterize architectural styles and patterns by their static and/or dynamic nature: The Layer pattern [9] is a static style because it is unusual that a concrete layered structure is changing, whereas the pipes & filters pattern is a dynamic style, because the actual Pipes and Filters [9] and their structure vary during execution.

Architectural Abstractions. Architectural abstractions over the software are (to a certain extent) dependent of the notation used for the software [8], but application domain independent.

Architectural abstractions can be supported by means of patterns [4], frameworks [7] and architecture notation/languages (AL's) [9]. Both pattern and framework technologies are powerful means of capturing abstractions over the software domain. Still the notation used for the representation of these is usual object-oriented notation — we simulate the architectural description. Only preliminary elements of specific architectural notation/languages are available, and the understanding of the necessary expressive power of such a notation/languages is mainly captured through architectural styles (pattern-like abstractions at architecture level) [9] and [3].

Characterization: Notation of Architecture Design.

- *Architecture notation can be informal.* Any notation supports thinking of and description of solutions and models, but it also introduces limitations. To be innovative we need expressive freedom, for example by use of informal notation. In practice we seem to prefer complete/formal specifications, and we need to include all details also concerning architecture. Only with a formal notation the power of reverse engineering can possibly be obtained.
- *Architecture notation can be several notations.* During the innovation of a model several notations can be applied simultaneously, possibly to various partial models such that each of these models can be expressed in a special notation (for example type of diagrams). In addition, models can be created from different perspectives — often overlapping and containing redundant information. Redundancy supports understanding. In practice several notations and corresponding models introduce an inconsistency problem. It is much more convenient with one unifying (but insufficient) notation, and it is much more straightforward to handle one unifying model. By different perspectives we usually introduce conflicting models to be resolved, and ultimately we have to choose among perspectives, or go through a complicated process to integrate and unify perspectives.

3.3: Principles.

Characterization: Principles of Architecture Design.

- *Architecture must be developed explicitly.* The elements included in methodologies for architecture design are still preliminary. The result of the design is not described explicitly. Architectural design is still immature — our understanding of abstraction over software is still insufficient. The evolution process where architectural abstractions and abstraction mechanisms of architectural languages are developed further by mutual influence has not really started — it has started with architectural styles

and patterns, but languages are still not available. In practice we mostly describe architecture implicitly as an integrated part of other models. When it is actually explicitly described/illustrated, it is usually done in object-oriented languages, as it is the case for design patterns and frameworks. Architectural abstractions are typically related to problem and the usage domains. The abstract world over the software domain is not explored in praxis.

- *Architecture design must be supported by tools.* Principles are usually (at least partially) supported by tools. Tools rely on notation. Because there is no unifying notation and the notation is partially informal, etc. tools for architecture design are not very powerful — they are more like simple drawing tools. In practice tools constitute the principles (and define praxis) in many organizations. Tools are important for process, notation and documentation standards.

4: Summary

Design of software architecture is presented as abstraction over the software domain. Given this perspective we characterize architecture according to process, notation and principles. This theoretical characterization represents a research point of view of architecture design. The main characteristics of architecture in this article are: 1) architecture is a model of the software domain, 2) architecture makes explicit some properties of the software and lets us reason about these properties, 3) architecture is in principle independent of the problem and usage domains.

References

[1] L. Bass, P. Clements, R. Kazman, K. Bass. Software Architecture in Practice. Addison-Wesley, 1998.

[2] G. Booch, J. Rumbaugh, I. Jacobsen. The Unified Modeling Language User Guide. Addison-Wesley, 1999.

[3] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, M. Stal. Pattern-Oriented Software Architecture: A System of Patterns. Wiley & Sons, 1996.

[4] E. Gamma, R. Helm, R. Johnson, J. Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

[5] E. E. Jacobsen, B. B. Kristensen, P. Nowack: Models, Domains and Abstraction in Software Development. Proceedings of International Conference on Technology of Object-Oriented Languages and Systems, 1998.

[6] E. E. Jacobsen, B. B. Kristensen, P. Nowack, T. Worm: Software Evolution: Prototypical Deltas. Proceedings of International Conference on Technology of Object-Oriented Languages and Systems, 1999.

[7] R. E. Johnson, B. Foote: Designing Reusable Classes. Journal of Object-Oriented Programming, 1988.

[8] B. B. Kristensen. Architectural Abstractions and Language Mechanisms. Proceedings of the Asia Pacific Software Engineering Conference, 1996.

[9] M. Shaw, D. Garlan: Software Architecture: Perspectives on an Emerging Discipline. Prentice-Hall, 1996.

[10] C. Szyperski: Component Software. Beyond Object-Oriented Programming. ACM Press/Addison Wesley, 1998.